Learning USV Drift from Telemetry: Linear, LSTM and Transformer Models

Jake Cushway
University of Victoria
Victoria, Canada
jakecush1@uvic.ca

Rowan Hall
University of Victoria
Victoria, Canada
rowanhall@uvic.ca

Abstract

Between course corrections, the DataXplorer™ USV drifts with its motor off, and a pilot must step in before wind and current carry it out of its 10 nm station. We predict where the vessel will be one hour from now, using only its last hour of GPS track and wind. We define the task as regression and compare four models of increasing power (persistence, linear regression, LSTM, transformer), tested on a deployment they never saw. Code: https://github.com/jakecush1/autonomous-pilot-ML (`usv-intervention/v1_ml/rowan`).[1]

Keywords

Uncrewed surface vehicle, drift prediction, time series regression, LSTM, transformer

1 Introduction

The Open Ocean Robotics DataXplorer replaces a retired NOAA weather buoy and must stay within 10 nm of the buoy's old position. It mostly drifts with the motor off, and a shore pilot watches it around the clock although a correction is needed only every few hours. In Milestone 2 we proposed predicting *when* the pilot must act. The first building block is predicting *where* the drifting vessel is going: a predicted position an hour ahead can be checked against the boundary to warn the pilot early. Unlike drift models built on physics and weather forecasts [1, 5], we learn drift directly from the vessel's own logged telemetry. We contribute a formal task definition, a leak-free data pipeline, and a comparison of four models.

2 Problem Definition

In words: given the last hour of a drifting vessel's track and wind, predict how far it will move east and north in the next hour.

A deployment is a sequence of 1-minute telemetry records $\mathcal{S} = \{s_\tau\}_{\tau=1}^{T}$, where $s_\tau = (\phi_\tau, \lambda_\tau, v_\tau, \theta_\tau, m_\tau)$ holds latitude, longitude, wind speed (kn), the compass direction the wind comes *from*, and motor power (W). The vessel is *drifting* at minute τ if $m_\tau = 0$. Let $\mathbf{p}_\tau \in \mathbb{R}^2$ be its position in metres east and north of the station (§3.1). For a time t at which the vessel has drifted for the past H minutes, our solution is a function

$$f : \mathbb{R}^{H\times 4} \to \mathbb{R}^2, \qquad \hat{\mathbf{y}}_t = f(\mathbf{X}_t) \approx \mathbf{y}_t = \mathbf{p}_{t+H} - \mathbf{p}_t, \tag{1}$$

where $\mathbf{X}_t$ describes the past H minutes (4 numbers per minute) and $\mathbf{y}_t$ is the movement over the next H minutes, assuming the motor stays off. The predicted position is $\hat{\mathbf{p}}_{t+H} = \mathbf{p}_t + \hat{\mathbf{y}}_t$. The output is a distance, not a category, so this is regression rather than classification. We use $H = 60$.

[1] AI use (Guideline D): the code and a first draft of this paper were generated with Claude (Anthropic) and reviewed, run and edited by the authors.

3 Proposed Approach

Figure 1 shows the pipeline and Algorithm 1 gives the pseudo-code.

3.1 Representation

Position. GPS degrees are converted to metres from the station centre (ϕ_0, λ_0), so that distances and movements are meaningful: $\mathbf{p}_\tau = R\,\big((\lambda_\tau - \lambda_0)\cos\phi_0,\ \phi_\tau - \phi_0\big)$, with $R = 6371$ km and angles in radians. This flat-earth approximation is off by less than 0.3% within 10 nm. **Wind.** The wind blowing *from* direction θ pushes the vessel the opposite way, so we use the push vector $\mathbf{w}_\tau = -v_\tau(\sin\theta_\tau, \cos\theta_\tau)$ (east, north). Using sine and cosine also avoids the jump from 359° to 0°. **Input window.** For each minute we keep its movement $\Delta\mathbf{p}_\tau = \mathbf{p}_\tau - \mathbf{p}_{\tau-1}$ and the wind push, giving $\mathbf{X}_t = [\,(\Delta\mathbf{p}_\tau, \mathbf{w}_\tau)\,]_{\tau=t-H+1}^{t}$, a table of 60 rows by 4 columns.

3.2 Cleaning and Splitting the Data

Gaps of up to 5 missing minutes are filled by linear interpolation. A minute is *valid* if the motor is off, no value is missing, and the vessel moved less than 93 m (about 3 kn; faster means it was being towed, not drifting). A time t becomes a training sample $(\mathbf{X}_t, \mathbf{y}_t)$ only if all 120 minutes from $t - 59$ to $t + 60$ are valid, so neither the input nor the answer includes motor use. Five deployments (Apr–Oct 2026, 85,910 minutes) give 23,319 samples.

Samples one minute apart share 59 of their 60 input minutes, so a random train/test split would put near-copies of test samples in training. We therefore split by whole deployment: **train** (Apr, May–Jun, Aug; 16,471 samples), **validation** (Oct 3–13; 2,289), used only to decide when to stop training, and **test** (Oct 16–23; 4,559), never used for any choice. Inputs and targets are rescaled to mean 0 and standard deviation 1 using training data only.

3.3 Models

Each model is a different way to compute f in Eq. 1. **(1) Persistence** (baseline, no training): the next hour repeats the last, $\hat{\mathbf{y}}_t = \sum_\tau \Delta\mathbf{p}_\tau$. Drift is driven mostly by current and tide, which change slowly, so this is a strong bar to beat. **(2) Linear regression**: $\hat{\mathbf{y}}_t = \mathbf{A}\,\mathbf{z}_t + \mathbf{c}$, where $\mathbf{z}_t$ summarises the window in 6 numbers (past-hour movement, current wind push, mean wind push over the hour) and $\mathbf{A} \in \mathbb{R}^{2\times 6}$, $\mathbf{c}$ are fitted by least squares. It tests whether a fixed weighting of drift and wind helps, and its weights can be read directly. **(3) LSTM** [2]: reads the 60 rows of $\mathbf{X}_t$ one minute at a time, updating a memory vector $\mathbf{h}$; the final $\mathbf{h}_t$ is mapped to $\hat{\mathbf{y}}_t = \mathbf{W}\mathbf{h}_t + \mathbf{b}$. Unlike the 6-number summary, it can see whether the drift is turning or slowing down. **(4) Transformer** [6]: each minute becomes a 64-number vector plus a code for its position in time, $\mathbf{e}_\tau = \mathbf{W}_e\mathbf{x}_\tau + \mathrm{PE}(\tau)$. Self-attention layers let every minute

Table 1: Hyperparameters (top-level constants in the code).

LSTM	2 layers, 64 hidden units, dropout 0.2
Transformer	width 64, 4 heads, 2 layers, feed-forward 256, dropout 0.1
Training	Adam, learning rate 10^{-3}, batch 256, gradient clip 1.0
Stopping	≤80 epochs, patience 10 on validation loss; seed 0

Algorithm 1 Drift prediction: build data, train, predict

Input: deployments $\{\mathcal{S}_k\}$, horizon H, model f_Θ
Output: trained f_{Θ^*}; predicted position $\hat{\mathbf{p}}_{t+H}$

1: **for all** deployments $\mathcal{S}_k$ **do** ▷ build samples
2: fill gaps ≤ 5 min; $\mathbf{p}_\tau \leftarrow$ metres from centre
3: $\mathbf{w}_\tau \leftarrow -v_\tau(\sin\theta_\tau, \cos\theta_\tau)$ ▷ wind push
4: $\text{valid}_\tau \leftarrow$ motor off $\wedge$ no missing $\wedge\ \|\Delta\mathbf{p}_\tau\| < 93$
5: **for all** t where minutes $t{-}H{+}1..t{+}H$ are all valid **do**
6: add $(\mathbf{X}_t,\ \mathbf{y}_t = \mathbf{p}_{t+H} - \mathbf{p}_t)$ to the split of $\mathcal{S}_k$
7: rescale with train statistics; seed $\leftarrow 0$; best $\leftarrow \infty$
8: **for** epoch = 1..80 **do** ▷ train
9: **for all** mini-batches B of train **do**
10: $\Theta \leftarrow \text{Adam}\big(\Theta, \nabla_\Theta \frac{1}{|B|}\sum_B \|f_\Theta(\mathbf{X}) - \mathbf{y}\|^2\big)$
11: **if** val loss < best **then** best $\leftarrow$ val loss; $\Theta^* \leftarrow \Theta$
12: **else if** no improvement for 10 epochs **then break**
13: **return** $\hat{\mathbf{p}}_{t+H} = \mathbf{p}_t + f_{\Theta^*}(\mathbf{X}_t)$ ▷ predict

compare itself with every other; the outputs are averaged over the hour and mapped to $\hat{\mathbf{y}}_t$.

The models go from simplest to most flexible; a more complex one is only worth using if it beats the simpler one on unseen data. Sizes are in Table 1.

3.4 Training, Inference and Scoring

The LSTM and transformer minimise mean squared error with Adam [3]. After each pass over the training data (epoch) we keep the weights with the lowest validation loss and stop after 10 epochs without improvement. This matters because the 23k overlapping windows hold only about 400 independent hours of drift, and both networks start memorising within a few epochs. Settings are common defaults, not tuned; tuning is part of Milestone 4. At inference the output is scaled back to metres and added to $\mathbf{p}_t$.

We score each prediction by its **endpoint error** $e_t = \|\hat{\mathbf{p}}_{t+H} - \mathbf{p}_{t+H}\|_2$, the distance in metres between predicted and true position, since distance is what matters to the pilot. We report the median (a typical miss, robust to GPS glitches) and the 90th percentile (a bad-case miss, usable as a safety radius).

Worked example. On Oct 19 at 21:15 (test deployment) the vessel had moved $(774, 555)$ m east/north in the past hour, slowing down, with a light 4 kn wind. Persistence predicts the same $(774, 555)$ m again; the trained LSTM predicts $(460, 353)$ m; the vessel actually moved $(283, 194)$ m. The endpoint errors are 609 m for persistence and 238 m for the LSTM, which noticed the slowdown.

4 Implementation

The code is in Python (scikit-learn, PyTorch), one file per pipeline stage, matching Algorithm 1: `data_prep.py` (representation, cleaning, split, endpoint error; lines 1–6), `linear_reg.py` (persistence, linear regression), `nn_models.py` (LSTM, transformer, training loop; lines 7–13) and `compare.py` (results table, chart). Outputs go to `outputs/`. The README covers the project, installation, quick start and results; dependencies are pinned in `requirements.txt` and the data are in the repository. The quick-start script `quickstart.sh` creates the environment on first use and runs all five steps in order (about 5 minutes on a laptop CPU):

```
git clone https://github.com/jakecush1/autonomous-pilot-ML
cd autonomous-pilot-ML/usv-intervention/v1_ml/rowan
bash quickstart.sh
```

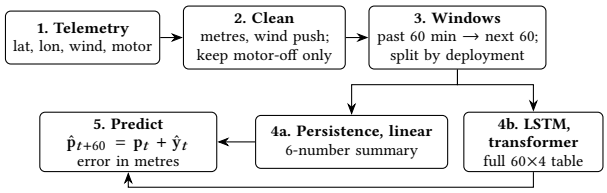


Figure 1: Pipeline: telemetry is cleaned and cut into 60-minute motor-off windows; each model maps a window to the next 60 minutes' movement.

Table 2: First run: endpoint error after 60 min, unseen test deployment.

Model	Median (m)	90th pct. (m)	R^2
No drift ($\hat{\mathbf{y}} = 0$)	586	1515	0.00
(1) Persistence	294	570	0.80
(2) Linear regression	318	631	0.77
(3) LSTM	247	497	0.85
(4) Transformer	**246**	**477**	**0.86**

Reproducibility. Seeds are fixed to 0 and the split is fixed by deployment, so the quick start on CPU reproduces Table 2 exactly (confirmed on three runs).

First results. Table 2 shows the pipeline runs end to end. Persistence already halves the error of assuming no drift. Linear regression does *not* beat persistence on the unseen deployment, while the LSTM and transformer both do, by about 16%, and are tied with each other. Milestone 4 will test more seeds and horizons and turn the prediction into a time until the vessel reaches the boundary [1], toward an intervention probability [4].

References

[1] Spencer August Dugan and Ingrid Bouwer Utne. 2024. Development of a Risk Indicator for Ship Drifting Groundings. In *Proc. PSAM17 & ASRAM2024.* https://www.iapsam.org/PSAM17/program/Papers/PSAM17&ASRAM2024-1377.pdf

[2] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (1997), 1735–1780. doi:10.1162/neco.1997.9.8.1735

[3] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proc. ICLR 2015.* https://arxiv.org/abs/1412.6980

[4] Susanna Dybwad Kristensen, Yiliu Liu, and Ingrid Bouwer Utne. 2022. Dynamic Risk Analysis of Maritime Autonomous Surface Ships. In *Proc. PSAM 16.* https://www.iapsam.org/PSAM16/papers/SU247-PSAM16.pdf

[5] Yuxuan Song et al. 2024. The Prediction and Dynamic Correction of Drifting Trajectory for Unmanned Maritime Equipment Based on Fully Connected Neural Network (FCNN) Embedding Model. *J. Mar. Sci. Eng.* 12, 12 (2024), 2262. doi:10.3390/jmse12122262

[6] Ashish Vaswani et al. 2017. Attention Is All You Need. In *Proc. NeurIPS 2017.* 5998–6008.